



OPEN SCIENCE

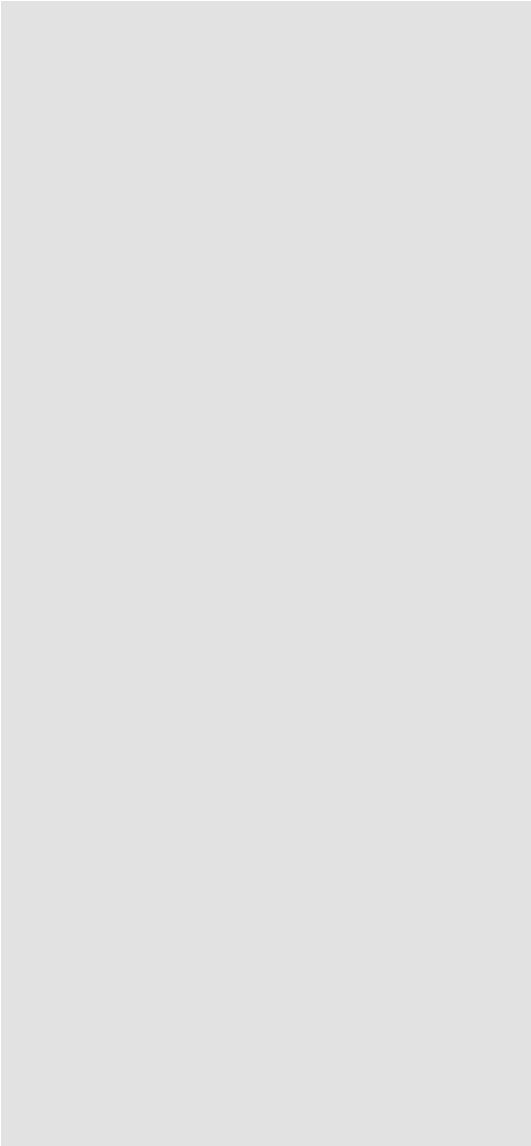
WORKFLOW:

ferramentas e práticas para pesquisa reprodutível com R e GitHub

Dra. Viviane Zulian

Programa de Pós-Graduação em Ecologia – UFSC

Maio de 2026





CRONOGRAMA

- Quarta-feira, 20 de maio, 9:00–12:00h (parte 1):

Por que usar Git/GitHub. Instalação e configuração do Git. Conexão com RStudio.

Criação de projetos (RStudio/GitHub). Licenças, ética e autoria no compartilhamento.

GIT *vs.* GITHUB



- Controla as versões dos códigos.
- Salva a história de alterações localmente (não na nuvem).



- Plataforma para visualizar versões criadas por Git.
- Nuvem para compartilhar, guardar e criar códigos.

TERMOS IMPORTANTES

commit – Ato de registrar (salvar) uma alteração realizada no código.

push – Ato de enviar a alteração realizada no código para a nuvem (GitHub).

pull – Ato de ‘baixar’ da nuvem a versão mais recente daquele código.

TERMOS IMPORTANTES

branch – Versão paralela do código, que não afeta a versão principal (*main branch*).

clone – Método para fazer download de um repositório do GitHub no seu computador.

fork – Novo repositório que compartilha código e configurações com o repositório do qual foi derivado.

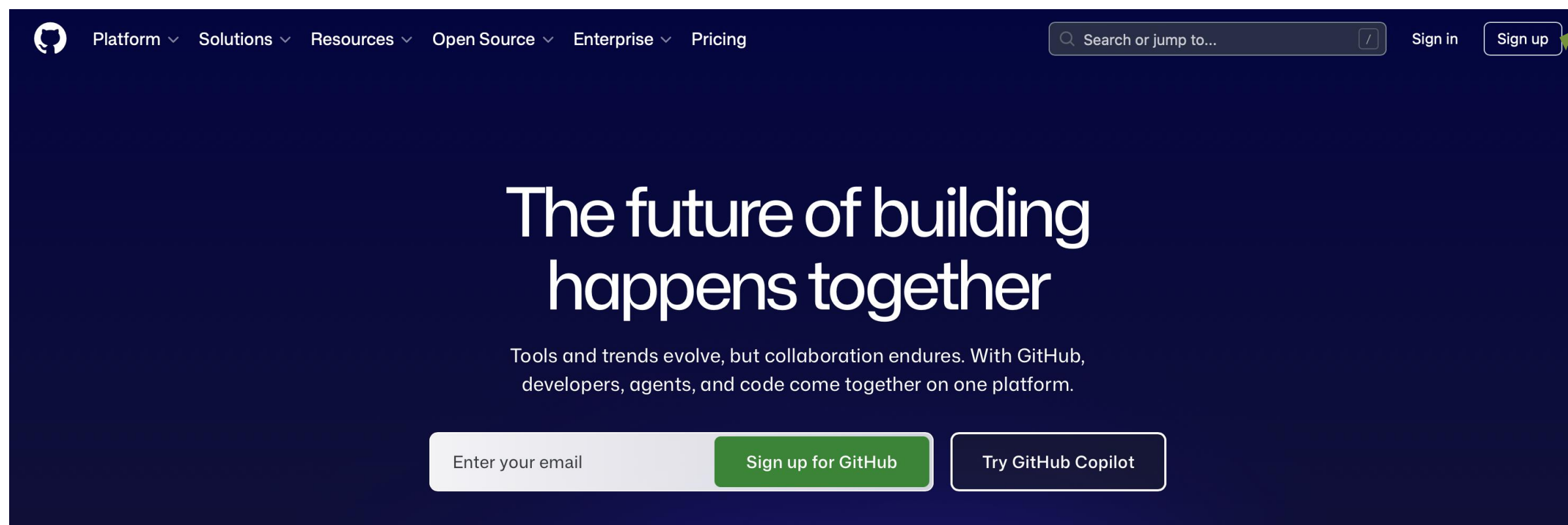
TERMOS IMPORTANTES

merge – Aplicar as modificações feitas em um *branch* em outro.
Juntar/mesclar as modificações.

pull request – Solicitação para mesclar alterações de um *branch* em outro.

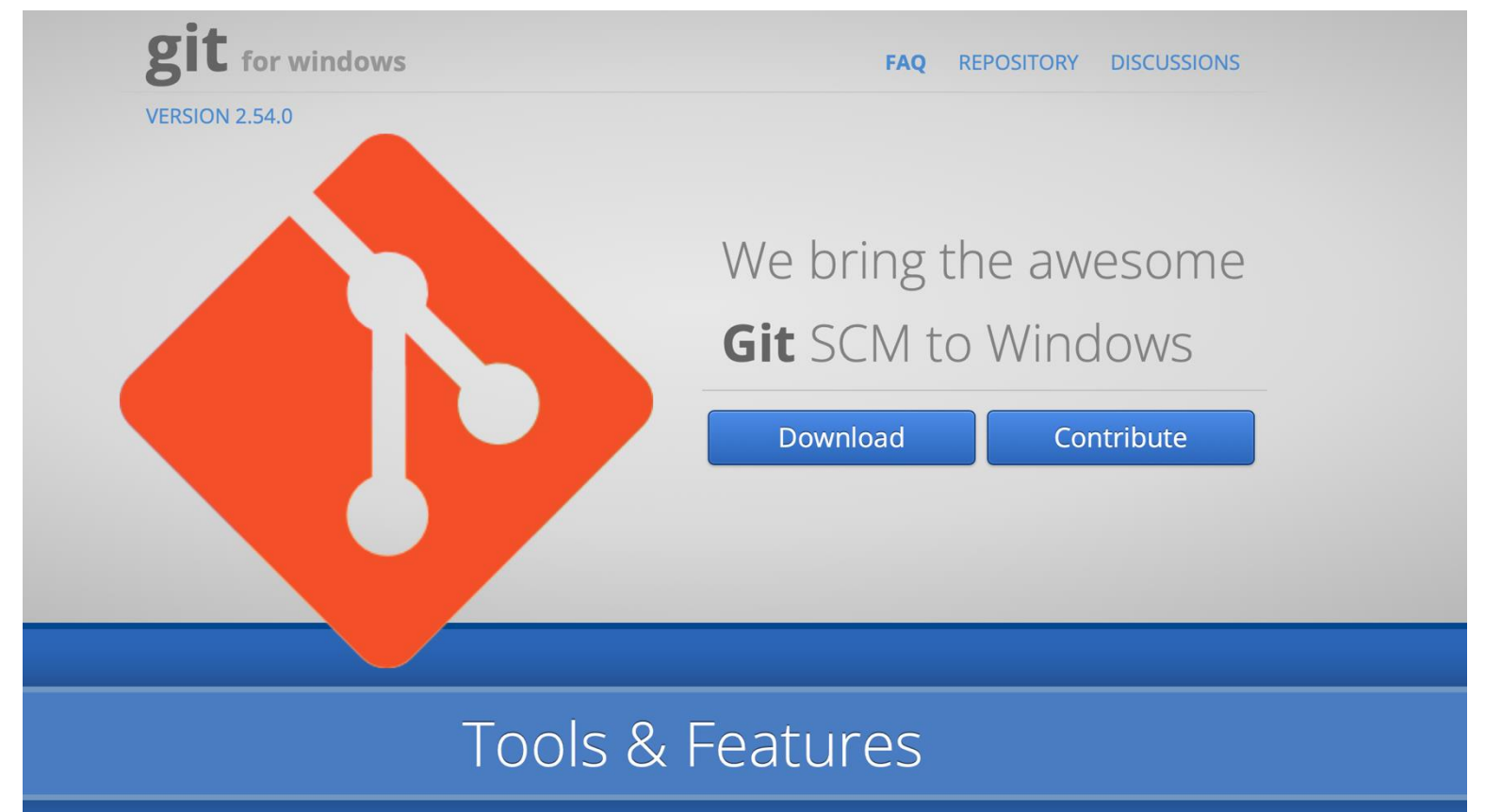
PRÁTICA

1. Criar um perfil na página do GitHub (<https://github.com>).



PRÁTICA

2. Baixar e instalar Git (<https://gitforwindows.org> ou <https://git-scm.com/install/mac>).

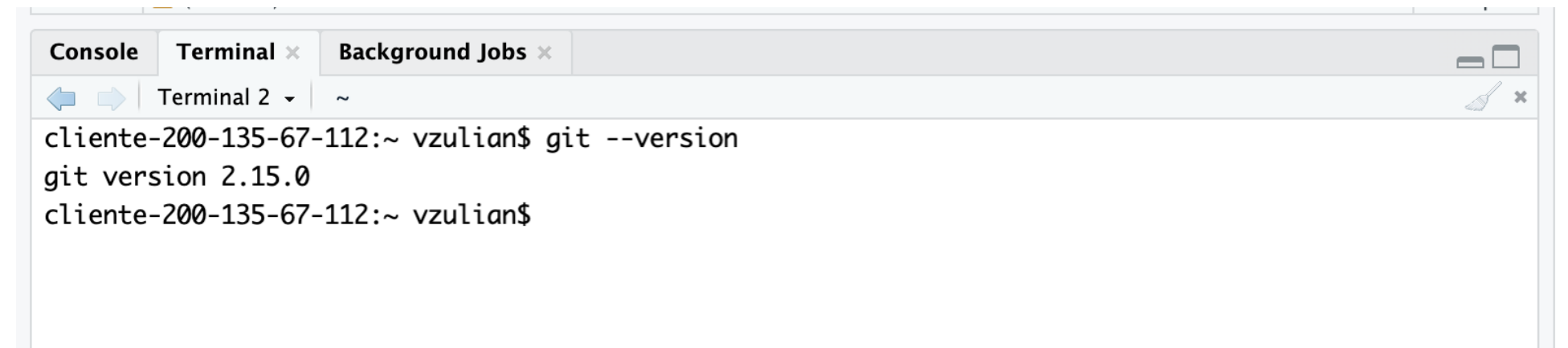


PRÁTICA

3. No Terminal do Rstudio: Conferir que Git está instalado

which git

git --version

A screenshot of the RStudio interface showing the Terminal pane. The terminal has tabs for 'Console', 'Terminal', and 'Background Jobs'. The 'Terminal' tab is active, showing a session with the prompt 'cliente-200-135-67-112:~ vzulian\$'. The command 'git --version' has been executed, and the output is 'git version 2.15.0'.

```
cliente-200-135-67-112:~ vzulian$ git --version
git version 2.15.0
cliente-200-135-67-112:~ vzulian$
```

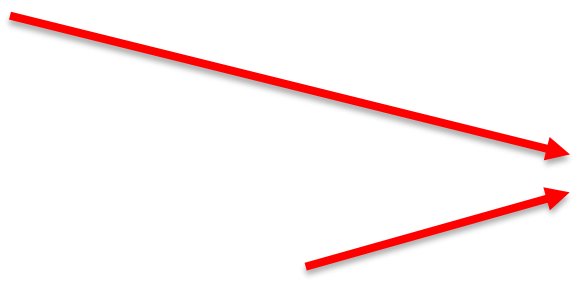
PRÁTICA

4. No Console do Rstudio: Configurar Git

```
install.packages('usethis')
```

```
library(usethis)
```

```
use_git_config(user.name = 'Viviane Zulian',  
               user.email = 'zulian.vi@gmail.com')
```



Use seu nome e e-mail
(os mesmos utilizados
no perfil do GitHub)



PRÁTICA

5. No Rstudio: Criar um projeto

Para poder usar Git e GitHub, precisamos estar trabalhando dentro de um projeto no Rstudio.

File -> New Project -> New Directory



PRÁTICA

6. No Console do Rstudio: Ativar Git

```
library(usethis)  
use_git()
```

Vai fazer com que um repositório seja criado. O RStudio vai reiniciar e agora uma aba chamada Git vai aparecer. Isso significa que tenho um repositório criado e que Git estará monitorando as versões.



PRÁTICA

6. No Rstudio: Criar código e usar commit

File -> New File -> R Script

Depois de criar um script, ele aparece na página Git. Agora, para que a alteração fique registrada, clico em *commit*, seleciono o arquivo, adiciono um comentário (*commit message*) e clico *commit*. Agora, quando olho o histórico de versões, vejo que está adicionado o script. Podemos editar o script pra ver o que acontece com o histórico de versões.



PRÁTICA

Até aqui:

- Git está funcionando (guardando as alterações).
- As alterações são para este projeto.
- As alterações ficam salvas no diretório, não na nuvem.

IMPORTANTE: Sempre que criamos um projeto novo, precisamos dizer para o RStudio para usar Git (repetir passo 6).

Instalar e configurar o Git novamente será necessário quando trocar ou formatar o computador.



PRÁTICA

7. No Console do Rstudio: Criar token no GitHub

Temos Git funcionando, mas ele ainda não está conectado com o GitHub. Para fazer isso, precisamos criar um token:

```
library(usethis)  
create_github_token()
```

Esse código vai abrir a página do GitHub, você precisa dar um nome para o token e gerar um. Copie o token.



PRÁTICA

8. No Console do Rstudio: Salvar minhas credenciais

```
install.packages(gitcreds)
```

```
library(gitcreds)
```

```
gitcreds_set()
```

Essa função vai pedir para colar o token que você criou.

Pronto! Agora o RStudio, Git e GitHub estão conectados. Entretanto, esse projeto específico seu não está conectado ao GitHub.



PRÁTICA

9. No Console do Rstudio: Conectar o GitHub ao meu projeto

```
library(usethis)  
use_github()
```

Pronto!

Agora o seu projeto aparecerá na sua página do GitHub e as funções de sincronização estarão ativas.



PRÁTICA

Clonar um repositório do GitHub para meu computador

Na página do repositório no GitHub:

Code -> HTTPS

Copie o HTTPS (link) do repositório.

No Rstudio, crie um novo projeto, mas escolha Version Control:

File -> New Project -> Version Control -> Git

Cole o link e crie o projeto.

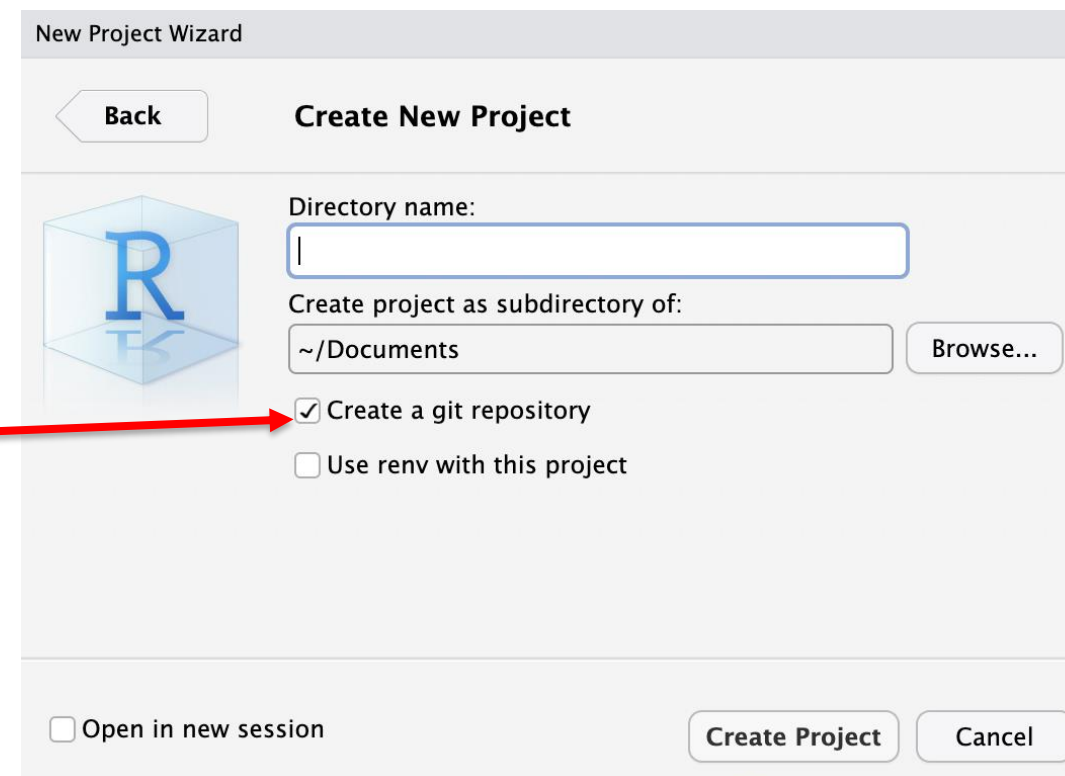
PRÁTICA

Criar um projeto no RStudio e um repositório do GitHub ao mesmo tempo

Depois de ter Git, GitHub e RStudio conectados, é possível criar um projeto e um repositório no GitHub ao mesmo tempo.

File -> New Project -> New Directory

Assinale 'Create a git repository'.





PRIVACIDADE

- Repositórios são criados como públicos, mas podem ser alterados para privados. (*Settings -> Change repository visibility*)
- Qualquer pessoa com perfil no GitHub poderá encontrar um repositório público.
- Em caso de projeto privado, somente pessoas adicionadas como colaboradoras terão acesso ao conteúdo. (*Settings -> Collaborators*)



Boas práticas

Sempre crie um README para o repositório descrevendo:

- O tema do projeto;
- Nome das pessoas autoras do projeto e códigos e contato;
- Fonte dos dados;
- Descrição do que encontrar em cada pasta.
- Informações sobre licença de uso e compartilhamento do material.



Boas práticas

Quando trabalhando em colaboração:

1. Abro o projeto e preciso clicar *Pull* para obter a versão mais recente, com todas as alterações que as pessoas colaboradoras realizaram.
2. Ao final do dia, faço *commit* das alterações que fiz e clico *Push*. Assim minhas alterações estarão salvas na nuvem e disponíveis para as pessoas colaboradoras.



Boas práticas

Para quais arquivos não devo usar controle de versões (*commit*)?

1. Planilhas de dados originais ou brutos, que sejam muito grandes (em Excel ou .txt);
2. Documentos em Word;
3. Output de modelos ou pastas com todos os resultados.

Aviso para Arquivos
maiores que 50 MiB

Bloqueio de Arquivos
maiores que 100 MiB

LICENÇAS E CITAÇÃO

<https://choosealicense.com/>

Choose an open source license

An open source license protects contributors and users. Businesses and savvy developers won't touch a project without this protection.

{ Which of the following best describes your situation? }



I need to work in a community.

Use the **license preferred by the community** you're contributing to or depending on. Your project will fit right in.

If you have a dependency that doesn't have a license, ask its maintainers to **add a license**.



I want it simple and permissive.

The **MIT License** is short and to the point. It lets people do almost anything they want with your project, like making and distributing closed source versions.

Babel, **.NET**, and **Rails** use the MIT License.



I care about sharing improvements.

The **GNU GPLv3** also lets people do almost anything they want with your project, *except* distributing closed source versions.

Ansible, **Bash**, and **uBlock Origin** use the GNU GPLv3.

- Para códigos, a licença mais comum é MIT (uso comercial, distribuição, modificação e uso público – condição: autores devem ser citados).
- Para dados, a mais comum é a CC BY 4.0 (Creative Commons Attribution 4.0 International) (uso comercial, distribuição, modificação e uso privado – condição: autores devem ser citados e todas as modificações realizadas nos dados devem ser documentadas).
- É possível adicionar o formato requerido de citação (mais comum para software ou pacotes).

PERGUNTAS?
